

FAQ

Lógica Booleana para Programadores

Perguntas e respostas diretas, com exemplos em Java, Python, JavaScript e SQL

Prof. Rômulo

Sumário

Módulo	Conteúdo
1	Fundamentos
2	Na prática do código
3	Simplificando e raciocinando
4	Armadilhas comuns
5	Indo além
6	Portas lógicas
7	Álgebra booleana e Mapa de Karnaugh
8	Exercícios com gabarito

Como usar: leia um card por vez, rode os exemplos no seu computador e tente prever o resultado *antes* de executar. No fim, resolva os exercícios do Módulo 8 e confira o gabarito.

MÓDULO 1

Fundamentos

1. O que é lógica booleana?

É o ramo da lógica que trabalha com apenas dois valores: **verdadeiro (true)** e **falso (false)**. Leva o nome de George Boole, matemático do século XIX. Toda decisão em um programa (if, while, filtros, validações) se apoia nela.

2. Por que o computador só usa dois valores?

Porque o hardware é feito de circuitos que distinguem dois estados: passa corrente ou não passa (1 ou 0). Os operadores lógicos são a versão em software das portas lógicas desses circuitos.

3. Quais são os operadores básicos?

Operador	Significado	Java / C / JS	Python	SQL
AND (E)	Verdadeiro só se ambos forem verdadeiros	&&	and	AND
OR (OU)	Verdadeiro se pelo menos um for verdadeiro		or	OR
NOT (NÃO)	Inverte o valor	!	not	NOT

Dica: o AND é exigente (todos precisam concordar); o OR é permissivo (basta um aceitar).

4. Como funcionam as tabelas-verdade?

Elas listam todas as combinações possíveis de entrada e a saída de cada operador:

A	B	A AND B	A OR B	A XOR B	NOT A
V	V	V	V	F	F
V	F	F	V	V	F
F	V	F	V	V	V
F	F	F	F	F	V

5. O que é XOR?

É o **ou exclusivo**: verdadeiro quando os dois valores são **diferentes**. Quase nenhuma linguagem tem um operador lógico xor para booleanos, mas você pode usar !=:

```
boolean temCafe = true, temChaNate = false;  
boolean escolheuUmSo = temCafe != temChaNate; // true
```

MÓDULO 2

Na prática do código

6. O que é avaliação de curto-circuito (short-circuit)?

O && e o || param de avaliar assim que o resultado já está definido:

- **falso && qualquerCoisa** dá falso, e o lado direito nem é executado.
- **verdadeiro || qualquerCoisa** dá verdadeiro, e o lado direito nem é executado.

```
if (lista != null && lista.size() > 0) { ... }
```

Se lista for null, o lista.size() nunca roda e você não leva um NullPointerException.

7. Qual a diferença entre && e & (e entre || e |)?

&& e || são operadores **lógicos**, com curto-circuito, sobre booleanos. & e | são **bit a bit** (bitwise), sobre os bits de inteiros.

```
int a = 6; // 110
int b = 3; // 011
System.out.println(a & b); // 2 (010)
System.out.println(a | b); // 7 (111)
```

Em Java, & e | também funcionam com booleanos, mas **sem curto-circuito**. Prefira && e || no dia a dia.

8. O que significam “truthy” e “falsy”?

Em Python e JavaScript, valores não booleanos também são tratados como verdadeiros ou falsos dentro de condições.

Linguagem	Valores falsy (todo o resto é truthy)
Python	False, None, 0, 0.0, "", [], {}, (), set()
JavaScript	false, 0, "", null, undefined, NaN

Em Java, C# e Go isso não existe: a condição **precisa** ser booleana.

Dica: em JavaScript, [] e {} são truthy, e a string "0" também é truthy.

9. Por que and e or em Python retornam valores, e não só True/False?

Porque eles retornam **um dos operandos**:

```
nome = "" or "Visitante" # "Visitante"
x = 5 and 10             # 10
y = 0 and 10             # 0
```

O or devolve o primeiro valor truthy (ou o último); o and devolve o primeiro valor falsy (ou o último). Útil para valores padrão, mas confunde iniciantes.

10. Qual é a precedência dos operadores lógicos?

Do mais forte para o mais fraco: **NOT**, depois **AND**, depois **OR**.

```
a || b && c // equivale a: a || (b && c)
```

Dica: na dúvida, use parênteses. Código legível vale mais que decorar tabela de precedência.

MÓDULO 3

Simplificando e raciocinando

11. O que são as Leis de De Morgan?

Elas ensinam a distribuir um NOT sobre AND e OR:

- **!(A && B)** é igual a **!A || !B**
- **!(A || B)** é igual a **!A && !B**

“Não é (maior de idade **e** tem documento)” equivale a “é menor de idade **ou** não tem documento”.

```
// Antes
if (!(idade >= 18 && temDocumento)) { negar(); }

// Depois (De Morgan)
if (idade < 18 || !temDocumento) { negar(); }
```

12. Como simplificar expressões booleanas?

Regra	Expressão
Identidade	$A \&\& \text{true} = A \cdot A \ \ \text{false} = A$
Dominação	$A \&\& \text{false} = \text{false} \cdot A \ \ \text{true} = \text{true}$
Idempotência	$A \&\& A = A \cdot A \ \ A = A$
Complemento	$A \&\& !A = \text{false} \cdot A \ \ !A = \text{true}$
Dupla negação	$!!A = A$
Absorção	$A \ \ (A \&\& B) = A$
Comutativa	$A \&\& B = B \&\& A$
Distributiva	$A \&\& (B \ \ C) = (A \&\& B) \ \ (A \&\& C)$

13. Posso evitar comparar booleano com true ou false?

Pode e deve.

```
// Evite
if (ativo == true) { ... }
return (idade >= 18) ? true : false;

// Prefira
if (ativo) { ... }
if (!ativo) { ... }
return idade >= 18;
```

14. Como dar bons nomes a variáveis booleanas?

Use nomes que formem uma pergunta de sim ou não: **isAtivo**, **temPermissao**, **podeEditar**, **estaLogado**. Evite nomes negativos como **naoEhValido**, porque `if (!naoEhValido)` é uma dupla negação confusa.

MÓDULO 4

Armadilhas comuns

15. Por que `if (x = 5)` é um erro tão comum?

Porque `=` é **atribuição** e `==` é **comparação**. Em C, `if (x = 5)` atribui 5 a x e a condição vira verdadeira, pois 5 é diferente de zero. Em Java o compilador reclama, porque um `int` não é booleano.

16. Por que $0.1 + 0.2 == 0.3$ dá falso?

Porque números de ponto flutuante são aproximações binárias:

```
print(0.1 + 0.2 == 0.3)          # False
print(abs((0.1 + 0.2) - 0.3) < 1e-9) # True
```

Compare com uma **tolerância** (ou use `math.isclose` em Python, `BigDecimal` em Java).

17. Por que $x > 1 \ \&\& \ x < 10$ funciona, mas $1 < x < 10$ nem sempre?

- Em **Python**, $1 < x < 10$ funciona (encadeamento de comparações).
- Em **Java**, **C** e **JavaScript**, vira $(1 < x) < 10$: compara um booleano com 10, e o resultado não é o esperado.

18. O que é a lógica de três valores do SQL (NULL)?

Em SQL, além de TRUE e FALSE existe o **UNKNOWN**, resultado de comparar algo com NULL:

```
SELECT * FROM alunos WHERE nota = NULL;  -- nao retorna nada!
SELECT * FROM alunos WHERE nota IS NULL;  -- correto
```

Expressão	Resultado
NULL = NULL	UNKNOWN
TRUE AND UNKNOWN	UNKNOWN
FALSE AND UNKNOWN	FALSE
TRUE OR UNKNOWN	TRUE

O WHERE só devolve as linhas em que a condição é TRUE.

MÓDULO 5

Indo além

19. Onde a lógica booleana aparece fora dos if?

- **Filtros** em listas, streams e consultas (`filter`, `WHERE`)
- **Permissões e flags** (`podeLer && !bloqueado`)
- **Máscaras de bits** (permissões Unix, `rwX`)
- **Buscas** (operadores AND, OR e NOT em motores de pesquisa)
- **Circuitos digitais** e **inteligência artificial** (regras, árvores de decisão)

20. Como treinar o raciocínio booleano?

1. Monte a tabela-verdade das expressões que você escreve.
2. Use De Morgan para reescrever condições complicadas.
3. Quebre condições longas em variáveis com bons nomes.
4. Escreva testes para os casos de borda (nulos, zero, limites).

```
boolean maiorDeIdade = idade >= 18;
boolean temDocumento = documento != null;
boolean podeEntrar = maiorDeIdade && temDocumento;
```

MÓDULO 6

Portas lógicas

21. O que são portas lógicas?

São pequenos circuitos eletrônicos que implementam os operadores booleanos. Um processador é feito de bilhões delas combinadas. Entender as portas ajuda a ver que, por baixo de qualquer programa, tudo se resume a AND, OR e NOT.

22. Quais são as 7 portas principais e suas saídas?

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

NOT é a oitava porta, com uma única entrada: NOT 0 = 1 e NOT 1 = 0. **NAND** é o AND invertido; **NOR** é o OR invertido; **XNOR** é o XOR invertido (dá 1 quando as entradas são **iguais**).

23. Por que NAND e NOR são chamadas de “universais”?

Porque qualquer outra porta pode ser construída usando **apenas NAND** (ou apenas NOR). Por isso chips costumam ser fabricados só com elas.

Quero	Com NAND
NOT A	NAND(A, A)
A AND B	NOT(NAND(A, B)) = NAND(NAND(A,B), NAND(A,B))
A OR B	NAND(NOT A, NOT B) = NAND(NAND(A,A), NAND(B,B))

24. Como as portas viram um somador? (meio-somador)

Somar dois bits A e B gera dois resultados: a **soma** (XOR) e o **vai-um**, ou carry (AND):

A	B	Soma (A XOR B)	Carry (A AND B)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Ou seja, $1 + 1 = 10$ em binário: soma 0 e vai-um 1. É assim que o computador soma.

MÓDULO 7

Álgebra booleana e Mapa de Karnaugh

Simplificar expressões de forma visual

25. O que é a forma de soma de produtos (SOP)?

É escrever a função como um OR de vários ANDs, um para cada linha da tabela-verdade em que a saída é 1 (os **mintermos**). Exemplo: $F = A'B + AB$, onde A' significa NOT A.

Dica: a notação A' é a mesma coisa que $\neg A$. Em álgebra booleana, AB significa A AND B e $+$ significa OR.

26. O que é um Mapa de Karnaugh e para que serve?

É uma tabela especial que organiza a tabela-verdade de modo que células **vizinhas** diferem em apenas um bit (ordem de Gray: 00, 01, 11, 10). Assim, dá para ver visualmente quais termos podem ser **agrupados e simplificados**, sem fazer contas.

27. Como simplificar com um mapa de 2 variáveis?

Função $F(A,B) = A'B + AB$ (saída 1 quando $B = 1$):

	B = 0	B = 1
A = 0	0	1
A = 1	0	1

Agrupando os dois 1 da coluna $B = 1$: o A muda (0 e 1) e some. Resultado: **$F = B$** .

28. Como simplificar com um mapa de 3 variáveis?

Função $F(A,B,C)$ com saída 1 nos mintermos 1, 3, 5 e 7. Colunas em ordem de Gray (BC):

A \ BC	00	01	11	10
0	0	1	1	0
1	0	1	1	0

Os quatro 1 formam um grupo (coluna 01 e 11, nas duas linhas). Dentro do grupo, A e B mudam e somem; só o **$C = 1$** permanece. Resultado: **$F = C$** .

```
// Em Java, as quatro condições se reduzem a uma:
boolean f = (!a && !b && c) || (!a && b && c) || (a && !b && c) || (a && b && c);
boolean fSimplificada = c;
```

29. Quais são as regras para agrupar no mapa?

1. Só agrupe células com valor **1**.
2. Os grupos têm tamanho **1, 2, 4, 8...** (potências de 2).
3. Grupos devem ser retângulos, e o mapa “dá a volta” nas bordas (a primeira e a última coluna são vizinhas).
4. Prefira os **maiores grupos possíveis**; cada 1 deve estar em pelo menos um grupo.
5. Cada grupo vira um termo; some os termos com OR.

MÓDULO 8

Exercícios com gabarito

Tente resolver sem olhar a página seguinte

1. Com $A = \text{true}$, $B = \text{false}$ e $C = \text{true}$, qual o resultado de $A \ \&\& \ (B \ || \ C)$?

Resposta: _____

2. Com $A = \text{true}$ e $B = \text{false}$, qual o resultado de $!(A \ \&\& \ B)$?

Resposta: _____

3. Reescreva usando De Morgan, sem o $!$ externo: $!(x > 5 \ || \ y < 3)$

Resposta: _____

4. O código abaixo lança erro de divisão por zero? Explique.

```
int x = 0; if (x != 0 && 10 / x > 2) { ... }
```

Resposta: _____

5. Em Python, qual o valor booleano de cada um? `bool([])`, `bool("0")`, `bool(0.0)`

Resposta: _____

6. O que retorna a consulta `SELECT * FROM alunos WHERE nota = NULL;`? Como corrigir?

Resposta: _____

7. Mostre pela tabela-verdade que $A \ \text{XOR} \ B$ é igual a $(A \ \text{OR} \ B) \ \text{AND} \ \text{NOT}(A \ \text{AND} \ B)$.

Resposta: _____

8. Simplifique: $A \ || \ (A \ \&\& \ B)$

Resposta: _____

9. Em Python, o que valem `0 or "0i"` e `5 and 10`?

Resposta: _____

10. Construa a porta NOT usando apenas uma porta NAND.

Resposta: _____

11. Use um Mapa de Karnaugh de 2 variáveis para simplificar $F(A,B) = A'B' + A'B$.

Resposta: _____

12. Escreva em Java uma expressão que seja verdadeira quando a pessoa tem entre 18 e 65 anos (inclusive) e possui documento.

Resposta: _____

1.	true. <code>B C = false true = true</code> ; depois <code>true && true = true</code> .
2.	true. <code>A && B = false</code> , e <code>!false = true</code> .
3.	x <= 5 && y >= 3. O NOT vira AND, e cada comparação é invertida (> vira <=, < vira >=).
4.	Não. Como <code>x != 0</code> é falso, o curto-circuito do && impede que <code>10 / x</code> seja avaliado.
5.	False, True e False. Lista vazia e 0.0 são falsy; a string "0" não é vazia, então é truthy.
6.	Nenhuma linha. <code>NULL = NULL</code> resulta em UNKNOWN. Correção: <code>WHERE nota IS NULL</code> .
7.	Verdadeiro. Para (V,V): $(V) \& (F) = F$; (V,F): $(V) \& (V) = V$; (F,V): $(V) \& (V) = V$; (F,F): $(F) \& (V) = F$. Coincide com a coluna XOR.
8.	A (lei da absorção).
9.	"Oi" e 10. O or devolve o primeiro truthy; o and devolve o último valor se nenhum for falsy.
10.	NAND(A, A). Se <code>A = 0</code> , dá 1; se <code>A = 1</code> , dá 0.
11.	F = A'. Os dois 1 ficam na linha <code>A = 0</code> ; B muda e some, sobrando A'.
12.	<code>idade >= 18 && idade <= 65 && temDocumento</code>